一、笔算二进制乘法

笔算乘法的分析

二进制乘法的笔算方法

$0.1101 = 1101 \times 2^{-4}$　　$0.1011 = 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} + 1 \times 2^{-4}$

$$0.1101 \times 0.1011 = 1101 \times 1 \times 2^{-8} + 1101 \times 1 \times 2^{-7} + 1101 \times 0 \times 2^{-6} + 1101 \times 1 \times 2^{-5}$$
$$= 1101 \times 2^{-8} + 1101 \times 2^{-7} + 0 \times 2^{-6} + 1101 \times 2^{-5}$$
$$= 0.00001101 + 0.0001101 + 0.000000 + 0.01101 = 0.10001111$$

被乘数　0.1101

乘数　× 0.1011

```
      1101
     1101
    0000
+  1101
-----------
0.10001111
```

笔算忽略不写

```
0.00001101
0.0001101   ← 位积向前错开一位
0.0000000   ← 位积向前错开一位
0.01101     ← 位积向前错开一位
+
-----------
0.10001111
```

二、机器实现

分析：乘法运算可写成加法运算，加法中的每一项又能写成科学计数法的形式，而此形式相当于移位运算，所以机器实现，需要两个方面：**加法**与**移位**

2.1 传统笔算的劣势

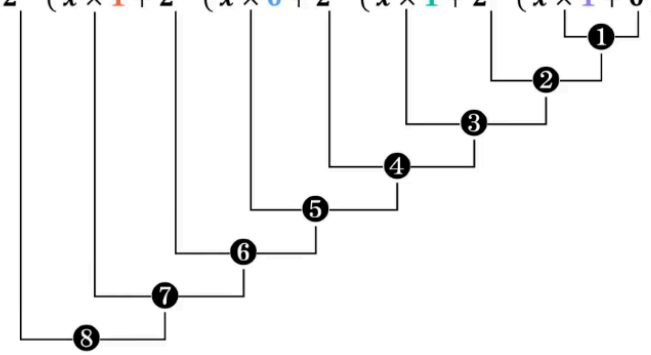
机器完全模仿笔算乘法步骤，所遇到的问题

- **运算过程中，产生的多个位积需要使用多个寄存器进行暂存，以便最后进行一次性相加。**
- **乘积的数据宽度增长了一倍，这将造成硬件资源浪费和运算时间的增加。**

2.2 优化后的方法

设 $x = 0.1101$，$y = 0.1011$，计算 $x \times y$。

$$x \times y = 2^{-1}(x \times 1 + 2^{-1}(x \times 0 + 2^{-1}(x \times 1 + 2^{-1}(x \times 1 + 0))))$$



共4次

1. 被乘数 x 加 0
2. 右移一位，得到 新的部分积

→ 先加法，后移位

3. 被乘数 x 加 部分积
4. 右移一位，得到 新的部分积

→ 先加法，后移位

5. 被乘数 0 加 部分积
6. 右移一位，得到 新的部分积

→ 先加法，后移位

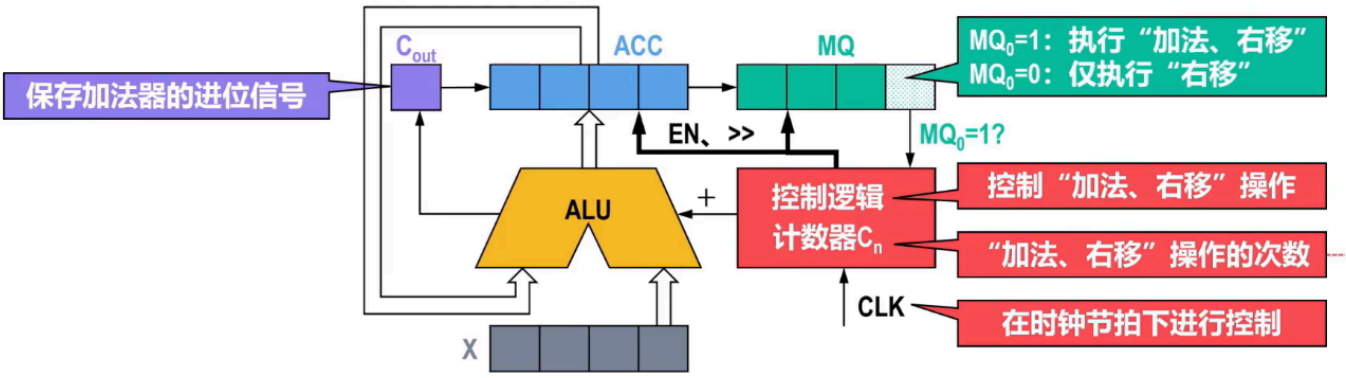
7. 被乘数 x 加 部分积
8. 右移一位，得到 最终乘积

→ 先加法，后移位

从最低位开始，每一步先计算**本位与上一个部分积的和**，然后**再移位**

需要一个位宽为4位的寄存器存放被乘数x；还需要两个位宽为4位的寄存器分别存放部分积和最终乘积的高位和低位，初始阶段高位寄存器被清0，低位寄存器存放乘数y。再配上加法器及其他相应电路，即可实现乘法运算。

2.3 基于机器实现的无符号数乘法的硬件逻辑实现



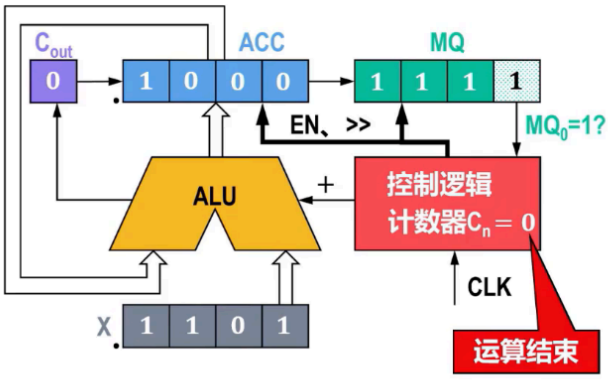
注：运算器组件的功能见[[1.6 计算机基本工作原理]]

示例

设$x=0.1101$，$y=0.1011$，计算$x\times y$。

$$x\times y = 2^{-1}(x\times 1+2^{-1}(x\times 0+2^{-1}(x\times 1+2^{-1}(x\times 1+0)))) = 0.10001111$$

全部完成



	加数 被乘数	进位 信号	被加数、和 乘积高位	乘数 乘积低位
	X	C_{out}	ACC	MQ
初始状态	1101	0	0000	1011
(X) + (ACC) → ACC	1101	0	1101	1011
(C_{out})、(ACC)、(MQ) >> 1	1101	0	0110	1101
(X) + (ACC) → ACC	1101	1	0011	1101
(C_{out})、(ACC)、(MQ) >> 1	1101	0	1001	1110
(C_{out})、(ACC)、(MQ) >> 1	1101	0	0100	1111
(X) + (ACC) → ACC	1101	1	0001	1111
(C_{out})、(ACC)、(MQ) >> 1	1101	0	1000	1111

计数器控制程序的终止，为0时，程序结束